

Giảm dư thừa và tối ưu hóa thuật toán

Hu Weidong

Nguồn gốc: Reducing redundancy and algorithm optimization

IOI China candidate team papers / 2004 / Hu Weidong.pdf

Tóm tắt

Trong các kỳ thi tin học, ta thường gặp hiện tượng dư thừa. Dư thừa làm giảm hiệu quả của thuật toán và chương trình ở nhiều mức độ: có khi ảnh hưởng rất nhỏ, nhưng cũng có khi làm độ phức tạp tăng lên đáng kể. Bài viết này tập trung vào trường hợp thứ hai, thông qua ví dụ để giải thích dư thừa ảnh hưởng đến hiệu quả thuật toán như thế nào và có thể giảm dư thừa ra sao.

Từ khóa. Dư thừa; tối ưu hóa thuật toán.

1 Mở đầu

Trong thi lập trình, một trong những mục tiêu ta theo đuổi là làm cho chương trình giải bài toán trong thời gian ít nhất, tức đạt hiệu quả cao nhất. Trong đời sống thực tế cũng vậy: người có hiệu suất cao thường có lợi thế trong cạnh tranh.

Dư thừa là các thao tác thừa hoặc lặp lại. Dư thừa có thể xuất hiện trong tìm kiếm, truy hồi, quy hoạch động và nhiều loại thuật toán khác. Theo định nghĩa đó, muốn thuật toán đạt hiệu quả cao nhất thì phải loại bỏ các xử lý dư thừa. Tuy nhiên, loại bỏ hoàn toàn dư thừa là rất khó; làm cho chương trình giải bài trong thời gian tuyệt đối nhỏ nhất cũng là việc khó đạt. Thông thường ta lùi một bước và đặt mục tiêu thực tế hơn: làm cho chương trình dùng ít thời gian nhất có thể. Nói cách khác, để nâng cao hiệu quả thuật toán, ta phải giảm các xử lý dư thừa trong thuật toán.

Muốn giảm dư thừa, cần liên tục tìm tòi và tích lũy kinh nghiệm qua quá trình làm nhiều bài. Dưới đây ta xét hai ví dụ cụ thể để xem dư thừa ảnh hưởng đến hiệu quả như thế nào và cách giảm nó.

2 Phân tích số nguyên

2.1 Mô tả bài toán

Cho số nguyên dương N . Hãy phân tích N thành tổng của một số số nguyên, mỗi số đều có dạng lũy thừa không âm của 2. Hỏi có bao nhiêu cách phân tích. Hai cách chỉ khác thứ tự các số hạng được xem là cùng một cách.

Ví dụ, với $N = 5$, ta có

$$5 = 1 + 1 + 1 + 1 + 1, \quad 5 = 1 + 1 + 1 + 2, \quad 5 = 1 + 2 + 2, \quad 5 = 1 + 4.$$

Vì vậy 5 có 4 cách phân tích.

2.2 Phân tích thô

Bài này có thể giải bằng truy hồi. Gọi $F[i, j]$ là số cách phân tích i thành tổng của các số sao cho số lớn nhất trong phân tích không vượt quá 2^j . Khi đó:

$$F[0, j] = 1,$$

$$F[i, 0] = 1,$$

vì nếu số lớn nhất không vượt quá $2^0 = 1$ thì chỉ có một cách: phân tích i thành i số 1.

Khi $i > 0$ và $j > 0$, các cách trong $F[i, j]$ chia thành hai loại:

- số lớn nhất đúng bằng 2^j , có số lượng $F[i - 2^j, j]$;
- số lớn nhất nhỏ hơn 2^j , có số lượng $F[i, j - 1]$.

Do đó

$$F[i, j] = F[i - 2^j, j] + F[i, j - 1].$$

Đáp án cần tính là $F[N, M]$, với M đủ lớn. Vì $i - 2^j \geq 0$, ta chỉ cần xét $j \leq \lfloor \log_2 i \rfloor$ và $1 \leq i \leq N$. Vì thế độ phức tạp thời gian là $O(N \log_2 N)$, và độ phức tạp bộ nhớ cũng là $O(N \log_2 N)$, nếu bỏ qua chi phí số lớn. Nhìn qua, độ phức tạp này đã khá thấp. Nhưng liệu có thể thấp hơn nữa không?

2.3 Giảm dư thừa

Để dễ nghiên cứu, trước hết xét trường hợp đặc biệt N là lũy thừa của 2, sau đó đưa trường hợp còn lại về trường hợp này.

2.3.1 Trường hợp là lũy thừa của hai

Giả sử $N = 2^M$, với M là số nguyên không âm. Để trực quan hơn, ta đặt mọi giá trị $F[i, j]$ lên các điểm nguyên của hệ tọa độ có trục ngang là I và trục dọc là J . Điểm có hoành độ i được gọi là điểm ở cột i , điểm có tung độ j được gọi là điểm ở hàng j ; giá trị $F[i, j]$ nằm tại cột i , hàng j .

Theo công thức truy hồi, nếu C là $F[i, j]$ thì hai giá trị góp vào nó là $A = F[i - 2^j, j]$ và $B = F[i, j - 1]$. Ta có thể hình dung có hai cạnh có hướng $A \rightarrow C$ và $B \rightarrow C$. Nếu nối tất cả các cạnh như vậy, ta nhận được một đồ thị phụ thuộc giữa các trạng thái.

Bây giờ xét mục tiêu $F[N, M]$. Nó bằng tổng của $F[N - 2^M, M] = F[0, M]$ và $F[N, M - 1]$. Tiếp theo, $F[N, M - 1]$ lại phụ thuộc vào $F[N - 2^{M-1}, M - 1]$ và $F[N, M - 2]$; $F[N, M - 2]$ phụ thuộc vào $F[N - 2^{M-2}, M - 2]$ và $F[N, M - 3]$; và cứ thế tiếp tục. Từ đó có thể thấy trong đồ thị đầy đủ có rất nhiều điểm mà tính hay không tính cũng không ảnh hưởng đến đáp án cuối cùng. Những điểm ấy là trạng thái dư thừa, và cũng không cần nối cạnh đến chúng.

Nếu xóa các trạng thái và cạnh dư thừa, chỉ giữ lại những trạng thái có thể ảnh hưởng đến điểm mục tiêu, đồ thị phụ thuộc trở nên gọn hơn rất nhiều. Câu hỏi là: rốt cuộc cần tính bao nhiêu điểm?

Quan sát cấu trúc còn lại cho ta phỏng đoán sau.

1. Khi $j = M - k$, hàng j cần và chỉ cần tính 2^k giá trị.
2. Những giá trị cần tính là

$$F[\lambda 2^j, j] \quad (1 \leq \lambda \leq 2^k).$$

Hai phỏng đoán này là đúng. Ta chứng minh bằng quy nạp.

Với $j = M$, hàng này chỉ cần tính $F[N, M]$, hiển nhiên đúng. Giả sử mệnh đề đúng tại hàng $j = M - k + 1$, tức các giá trị cần tính là

$$F[\lambda 2^j, j] \quad (1 \leq \lambda \leq 2^{k-1}).$$

Đặt $j' = j - 1 = M - k$. Khi cần tính $F[\lambda_0 2^j, j]$, công thức truy hồi buộc ta tính

$$F[\lambda_0 2^j, j'] = F[2\lambda_0 2^{j'}, j'].$$

Để tính giá trị này lại cần dùng $F[(2\lambda_0 - 1)2^{j'}, j']$, rồi tiếp tục lùi dần. Cuối cùng, mọi giá trị

$$F[x2^{j'}, j'] \quad (1 \leq x \leq 2\lambda_0)$$

đều cần được tính. Khi λ_0 chạy qua mọi giá trị có thể, hàng j' cần tính đúng các giá trị

$$F[x2^{j'}, j'] \quad (1 \leq x \leq 2^k),$$

phù hợp với phỏng đoán. Do đó hai mệnh đề đều đúng.

Số điểm thực sự hữu ích là

$$1 + 2 + 2^2 + \dots + 2^{M-1} = 2^M - 1 = N - 1,$$

trong khi cách ban đầu tính khoảng $N \cdot M$ điểm. Số trạng thái dư thừa lớn hơn rất nhiều so với số trạng thái bắt buộc phải tính. Nếu loại bỏ chúng, độ phức tạp thời gian có thể giảm xuống $O(N)$.

Vấn đề còn lại là xác định điểm nào cần tính. Có thể dùng trực tiếp mệnh đề trên, nhưng còn hai kết luận gọn hơn.

Kết luận 1. Trong mỗi cột, các điểm cần tính chắc chắn là một số điểm liên tiếp nằm ở phía dưới cùng.

Thật vậy, nếu cần tính $F[i, j]$, từ công thức truy hồi ta nhất định phải tính $F[i, j - 1]$, rồi $F[i, j - 2]$, tiếp tục cho đến $F[i, 1]$, còn $F[i, 0]$ đã biết. Vì vậy mọi điểm ngay bên dưới $F[i, j]$ đều phải được tính.

Kết luận 2. Khi $i = X$, số điểm cần tính trong cột i , ký hiệu T_i , bằng số chữ số 0 ở cuối biểu diễn nhị phân của X .

Chứng minh như sau. Giả sử biểu diễn nhị phân của i có đúng T_i chữ số 0 ở cuối.

Nếu $j \leq T_i$ thì $2^j \mid i$, tức tồn tại số nguyên λ sao cho $i = \lambda 2^j$. Vì $1 \leq i \leq N = 2^M$, ta có

$$1 \leq \lambda = \frac{i}{2^j} \leq \frac{2^M}{2^j} = 2^{M-j}.$$

Theo kết luận về các điểm hữu ích, $F[i, j]$ phải được tính. Đặc biệt, khi $j = T_i$, cột i có ít nhất T_i điểm cần tính.

Ngược lại, nếu $j \geq T_i + 1$ thì $2^j \nmid i$, không tồn tại số nguyên λ sao cho $i = \lambda 2^j$. Do đó $F[i, j]$ không cần tính. Vậy cột i chỉ cần đúng T_i điểm. Kết luận được chứng minh.

Như vậy, giảm độ phức tạp thời gian xuống $O(N)$ không còn là vấn đề.

Bây giờ xét bộ nhớ. Mọi dữ liệu không cần tính đều có thể không cấp phát. Giả sử khi cài đặt, vòng ngoài duyệt i tăng dần và vòng trong duyệt j tăng dần. Với trạng thái $F[i, j]$, hai giá trị cần dùng là $F[i, j - 1]$ và $F[i - 2^j, j]$. Trong các điểm đã tính của hàng $j - 1$, $F[i, j - 1]$ chính là điểm bên phải nhất hiện đã biết; trong các điểm đã tính của hàng j , $F[i - 2^j, j]$ cũng là điểm bên phải nhất hiện đã biết. Vì vậy, với mỗi j ta chỉ cần lưu giá trị $F[x, j]$ có x lớn nhất trong số các giá trị đã tính. Cách lưu này giảm lãng phí, giống tác dụng của mảng cuộn, và hạ độ phức tạp bộ nhớ xuống $O(\log_2 N)$.

2.3.2 Trường hợp không phải lũy thừa của hai

Giả sử

$$N = 2^M - r, \quad 2^{M-1} < N < 2^M.$$

Mục tiêu ban đầu là $F[N, M - 1]$. Vì

$$F[N, M] = F[N, M - 1] + F[N - 2^M, M],$$

mà $F[N - 2^M, M] = 0$ hoặc không tồn tại, ta có $F[N, M] = F[N, M - 1]$. Do đó có thể đổi mục tiêu thành $F[N, M]$.

Vẫn đặt các trạng thái $F[i, j]$ lên hệ tọa độ và xóa phần dư thừa như trên. Ta bổ sung thêm r cột giả

$$F[-r], F[-r + 1], F[-r + 2], \dots, F[-1],$$

với mọi giá trị trong các cột này bằng 0. Sau đó tịnh tiến toàn bộ các điểm sang phải r đơn vị. Bài toán thu được có dạng giống trường hợp $N' = 2^M$, chỉ khác là các cột từ 0 đến $r - 1$ đều có giá trị bằng 0, còn các cột còn lại vẫn thỏa quan hệ truy hồi của trường hợp lũy thừa của 2. Nhờ phép biến đổi này, ta có thể tính đáp án thuận lợi như trước.

2.4 Tiểu kết

Nhìn lại bài toán, cách làm ban đầu thực hiện rất nhiều thao tác dư thừa, khiến độ phức tạp thời gian và bộ nhớ cao hơn nhiều so với mức cuối cùng. Sau khi giảm dư thừa, cả hai độ phức tạp đều giảm mạnh. Vì vậy, giảm dư thừa là điểm then chốt trong tối ưu hóa bài này.

Liệu bài toán còn thuật toán tốt hơn không? Có công thức trực tiếp nào không? Điều đó vẫn cần tiếp tục khám phá.

3 Giá trị phần thưởng lớn nhất

3.1 Mô tả bài toán

Là người đạt giải thưởng tiêu dùng cao nhất trong năm, bạn có cơ hội tham gia một trò quay thưởng khuyến mãi. Trò chơi diễn ra trên $N + 2$ bậc thang, đánh số từ 0 đến $N + 1$. Trên mỗi bậc từ 1 đến N có một phần thưởng. Nếu bạn đi đúng M bước, với $M \leq N + 1$, từ bậc 0 đến đúng bậc $N + 1$, và không đi ra ngoài $N + 2$ bậc này, thì bạn nhận được các phần thưởng trên những bậc đã đi qua. Nếu không, bạn không nhận được gì. Trước hết, bạn gán cho mỗi phần thưởng một điểm số dương. Bạn muốn tổng điểm của các phần thưởng nhận được là lớn nhất.

Dĩ nhiên, mỗi bước không thể đi quá nhiều bậc, vì như vậy có thể bị ngã. Giả sử mỗi bước đi ít nhất 0 bậc, tức đứng yên, và nhiều nhất K bậc. Nói cách khác, từ bậc i có thể đi theo chiều tăng nhân đến tối đa bậc $i + K$, hoặc theo chiều giảm nhân đến tối đa bậc $i - K$.

3.2 Mô hình toán học

Bài toán có thể được biểu diễn như sau. Cho dãy số nguyên

$$a_0, a_1, a_2, \dots, a_N, a_{N+1},$$

trong đó $a_0 = a_{N+1} = 0$ và $a_1, a_2, \dots, a_N > 0$. Với $M < N$ và K , hãy tìm một dãy con

$$a_{i_0}, a_{i_1}, a_{i_2}, \dots, a_{i_M}$$

sao cho:

1. $0 = i_0$ và $i_M = N + 1$;
2. $-K \leq i_1 - i_0 \leq K, -K \leq i_2 - i_1 \leq K, \dots, -K \leq i_M - i_{M-1} \leq K$;
3. tổng $a_{i_0} + a_{i_1} + \dots + a_{i_M}$ là lớn nhất.

3.3 Phân tích thô

Đề bài chỉ nói mỗi bước đi được nhiều nhất bao nhiêu bậc, không cấm đi lùi. Vậy đi lùi có ý nghĩa không? Giả sử có một bước đi lùi, tức tồn tại p sao cho $i_{p+1} < i_p$. Chọn p nhỏ nhất thỏa điều kiện này. Khi đó

$$0 \leq i_p - i_{p+1} \leq K, \quad -K \leq i_{p-1} - i_p \leq 0.$$

Suy ra

$$-K \leq i_{p-1} - i_{p+1} \leq K.$$

Nếu đổi thứ tự i_p và i_{p+1} , tức biến đoạn i_{p-1}, i_p, i_{p+1} thành i_{p-1}, i_{p+1}, i_p , các ràng buộc vẫn được thỏa mãn và tổng điểm không đổi. Lặp thao tác này, ta có thể đưa dãy về dạng

$$i_0 \leq i_1 \leq i_2 \leq \dots \leq i_M.$$

Vì ngoài $a_0 = a_{N+1} = 0$, mọi điểm số đều dương, dừng lại trên một bậc đã đi qua không tốt bằng đi đến một bậc chưa đi qua. Do đó ta có thể tiếp tục giả sử

$$i_0 < i_1 < i_2 < \dots < i_M.$$

Sau xử lý trên, bài toán có cấu trúc con tối ưu và có thể giải bằng quy hoạch động. Gọi $F[i, j]$ là tổng lớn nhất của dãy con khi số được chọn thứ i là a_j . Khi đó

$$F[i, j] = \max_{j-K \leq x < j} \{F[i-1, x]\} + a_j.$$

Số trạng thái là $N \cdot M$, mỗi trạng thái có K quyết định, nên độ phức tạp thời gian là $O(NMK)$. Thuật toán này có thể tối ưu tiếp không?

3.4 Giảm dư thừa

Khi tính $F[i, j-1]$ và $F[i, j]$, ta lần lượt cần tính

$$\max_{j-K-1 \leq x < j-1} \{F[i-1, x]\} \quad \text{và} \quad \max_{j-K \leq x < j} \{F[i-1, x]\}.$$

Trong đó

$$\max_{j-K-1 \leq x < j-1} \{F[i-1, x]\} = \max \left(F[i-1, j-K-1], \max_{j-K \leq x < j-1} \{F[i-1, x]\} \right),$$

và

$$\max_{j-K \leq x < j} \{F[i-1, x]\} = \max \left(\max_{j-K \leq x < j-1} \{F[i-1, x]\}, F[i-1, j-1] \right).$$

Có thể thấy đoạn giữa

$$\max_{j-K \leq x < j-1} \{F[i-1, x]\}$$

được tính lặp lại. Nếu giảm được phép tính dư thừa này, độ phức tạp sẽ giảm. Thông thường có ba loại cách xử lý.

Cách thứ nhất. Giả sử khi tính $F[i, j - 1]$, giá trị lớn nhất nhận được là X . Khi tính $F[i, j]$, so sánh X với $F[i - 1, j - K - 1]$. Nếu hai giá trị không bằng nhau thì chắc chắn

$$X = \max_{j-K \leq x < j-1} \{F[i - 1, x]\},$$

nên giá trị lớn nhất khi chuyển sang $F[i, j]$ là

$$\max\{X, F[i - 1, j - 1]\},$$

và có thể chuyển trong $O(1)$. Nhưng nếu $X = F[i - 1, j - K - 1]$, ta buộc phải tính lại

$$\max_{j-K \leq x < j} \{F[i - 1, x]\},$$

tốn $O(K)$. Với dữ liệu đặc biệt, chẳng hạn $a_1 > a_2 > a_3 > \dots > a_N$, tổng độ phức tạp vẫn là $O(NMK)$. Cách này chưa tối ưu từ gốc.

Cách thứ hai. Dùng cấu trúc dữ liệu như heap hoặc cây đoạn để tối ưu truy vấn cực trị trên cửa sổ. Khi đó độ phức tạp thời gian có thể giảm xuống $O(NM \log_2 K)$, nhưng độ phức tạp lập trình tăng lên.

Cách thứ ba. Phân tích trực tiếp toàn bộ quá trình chuyển trạng thái có vẻ khó, nên ta xét riêng bước chuyển từ lớp $F[i - 1]$ sang lớp $F[i]$. Chú ý rằng nếu $a < b < j$ và

$$F[i - 1, b] \geq F[i - 1, a],$$

thì khi tính $F[i, j]$, vì đã có $F[i - 1, b]$, giá trị lớn nhất sẽ không cần lấy $F[i - 1, a]$. Nếu hai giá trị bằng nhau và cùng là lớn nhất, ta có thể chọn $F[i - 1, b]$ thay cho $F[i - 1, a]$. Vì vậy, khi tính $F[i, j]$, trạng thái $F[i - 1, a]$ là dư thừa và không cần liệt kê.

Nếu dùng một danh sách tuyến tính để lưu mọi trạng thái của $F[i - 1]$ còn cần liệt kê, sắp theo chỉ số tăng dần, thì giá trị của các trạng thái trong danh sách sẽ đơn điệu giảm. Khi đó giá trị lớn nhất cần tìm chính là giá trị của phần tử đầu danh sách. Để cài đặt hiệu quả, ta biến danh sách này thành một hàng đợi hai đầu đơn điệu.

Khi j tăng, mọi trạng thái $F[i - 1, x]$ với $x < j - K$ phải bị xóa khỏi danh sách. Vì j chỉ tăng từng đơn vị, mỗi lần nhiều nhất cần xóa phần tử đầu. Sau đó chèn $F[i - 1, j - 1]$ vào cuối danh sách. Trong quá trình chèn, để giữ tính đơn điệu giảm, nếu một phần tử cuối $F[i - 1, x]$ nhỏ hơn $F[i - 1, j - 1]$, ta xóa nó khỏi cuối danh sách; tiếp tục như vậy cho đến khi có thể chèn phần tử mới.

Trong cấu trúc này, mỗi trạng thái $F[i - 1, x]$ với $1 \leq x \leq n$ nhiều nhất được đưa vào hàng đợi một lần và bị lấy ra một lần. Truy vấn phần tử lớn nhất tốn $O(1)$. Vì vậy tổng độ phức tạp thời gian giảm xuống $O(NM)$.

Ví dụ, với

$$F[i - 1] = (6, 3, 5, 2, 8), \quad K = 3,$$

quá trình duy trì hàng đợi có thể viết như sau:

j	Thao tác	Hàng đợi	Lớn nhất
0	Bắt đầu	$[\]$	—
1	Chèn 6	$[6]$	6
2	Chèn 3	$[6, 3]$	6
3	Chèn 5, xóa 3 ở cuối	$[6, 5]$	6
4	Xóa 6 ở đầu; chèn 2	$[5, 2]$	5
5	Chèn 8, xóa 2 và 5 ở cuối	$[8]$	8

3.5 Tiểu kết

Trong bài này, dư thừa không dễ loại bỏ như ở bài phân tích số nguyên. Khi đó ta có thể cân nhắc dùng cấu trúc dữ liệu. Đồng thời, cấu trúc dữ liệu không phải là thứ bất biến; nếu biết vận dụng linh hoạt, nó có thể phát huy tác dụng rất lớn. Trong quá trình giảm thao tác dư thừa, ta thường cần đến cấu trúc dữ liệu.

4 Tổng kết

Từ các ví dụ trên có thể thấy rằng trong thiết kế thuật toán và lập trình, dư thừa là điều khó tránh khỏi. Dư thừa là kẻ thù của hiệu quả; giảm dư thừa chắc chắn có thể cải thiện đáng kể hiệu suất của thuật toán và chương trình. Trong ví dụ thứ nhất, độ phức tạp thời gian giảm từ $O(N \log_2 N)$ xuống $O(N)$, độ phức tạp bộ nhớ giảm từ $O(N \log_2 N)$ xuống $O(\log_2 N)$. Trong ví dụ thứ hai, độ phức tạp thời gian giảm từ $O(NMK)$ xuống $O(NM)$.

Không có định lý hay công thức cố định nào có thể áp dụng máy móc để giảm dư thừa. Chỉ bằng cách phân tích nghiêm túc, chịu khó tìm tòi và tích lũy kinh nghiệm qua quá trình làm bài, ta mới tìm được những phương pháp tốt để giảm dư thừa.

Lời cảm ơn

Trong quá trình viết bài, các thầy Xiang Qizhong, thầy Li và Jin Kai đã giúp đỡ tôi rất nhiều; tôi xin chân thành cảm ơn. Tôi cũng chân thành cảm ơn Ren Kai, Wang Jun, Yi Wei, Kang Lianghuan, Zheng Zhao và nhiều bạn khác, những người vẫn dành thời gian đọc bài và góp nhiều ý kiến quý báu khi kỳ thi cuối kỳ đã đến gần. Cuối cùng, xin cảm ơn các giám khảo và thầy cô đã bớt thời gian tổ chức hoạt động này.

Tài liệu tham khảo

Xiang Rongjing, *Tận dụng đầy đủ tính chất bài toán: phân tích tối ưu “cá thể hóa” trong quy hoạch động*, luận văn năm 2003.